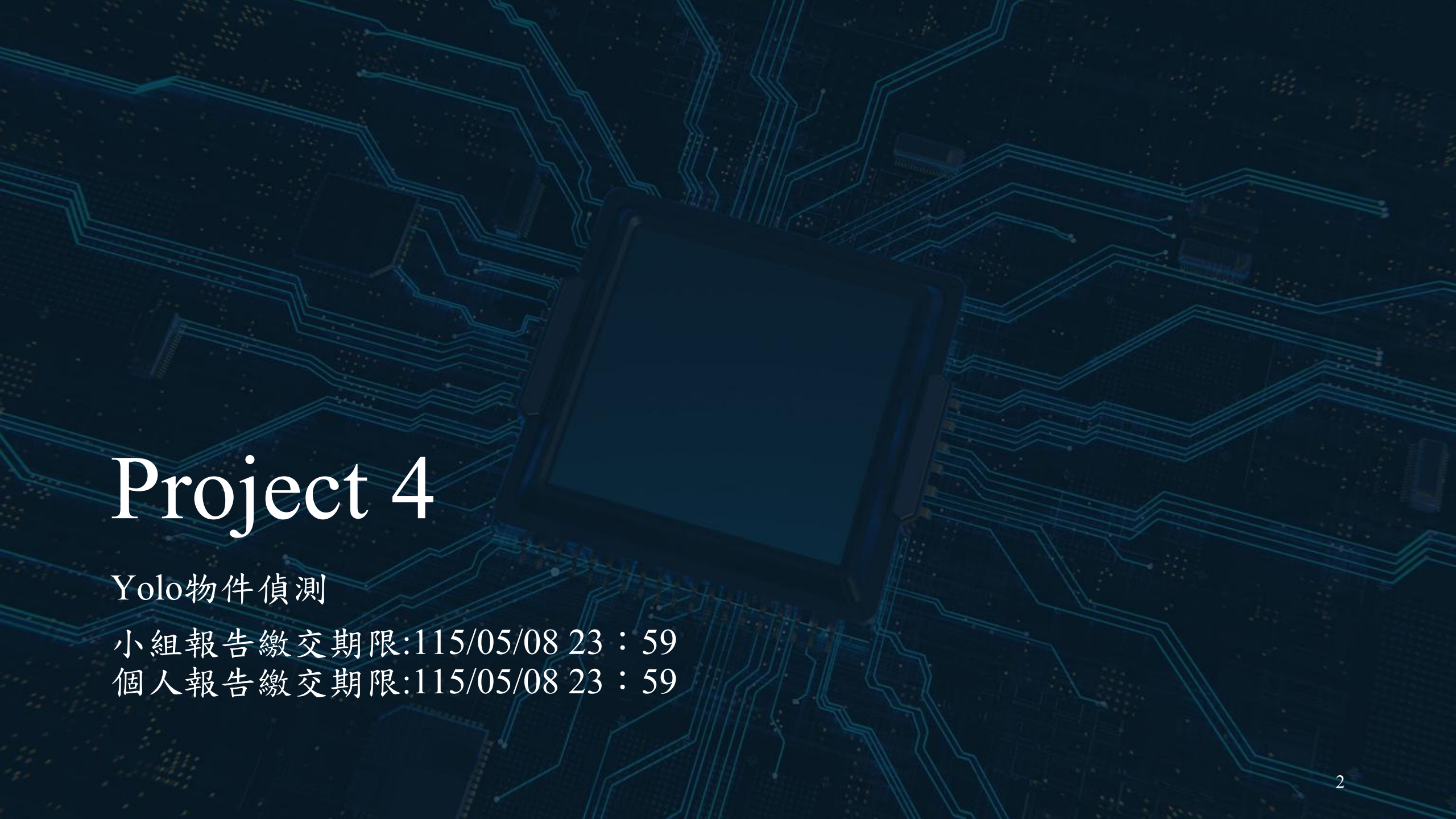




多媒體技術與應用

Spring, 2026.

Instructor : Yen-Lin Chen(陳彥霖), PH.D.
Distinguished Professor
Dept. Computer Science and Information Engineering
National Taipei University of Technology



Project 4

Yolo物件偵測

小組報告繳交期限:115/05/08 23 : 59

個人報告繳交期限:115/05/08 23 : 59

YoloV4應用與實作



物件辨識於邊緣智慧計算應用

- **影像處理與分析**

- 影音編輯：利用 YOLO 進行物件識別和追蹤，可以自動化許多影音編輯任務，如場景切換的智慧識別，自動標記和分類影音內容。
- 內容過濾：YOLO 可以用於即時識別和過濾不適宜的內容，確保符合廣播標準和規範，這在直播平台尤其重要。

- **即時視訊分析應用**

- 安全監控：在安全監控系統中，YOLO 可以即時識別和追蹤人物和物體，快速響應可疑行為，提高安全性。
- 體育賽事分析：在體育轉播中，YOLO 可以用於追蹤球員和球的位置，自動生成遊戲統計資料和重點回放，豐富觀眾的觀賽體驗。



應用與實作



前置作業 - 如何創建和標記自己的資料集

- 收集圖片資料
- 進行相片標註 (LabelImg、Roboflow)
- 資料預處理



收集圖片資料 (道路偵測)

收集具有豐富多樣性和實際環境變化的圖片

- **不同路況**：收集包括高速公路、城市道路、郊區道路等不同類型的道路圖片。
- **不同時間**：包括白天和夜晚的圖片，以捕捉不同照明條件下的道路情況。
- **不同氣候條件**：收集晴天、雨天、陰天等不同氣候下的道路圖片。
- **道路標誌和標線**：確保圖片中包含各種道路標誌和道路標線，這對於道路偵測模型來說非常重要。
- **不同交通狀況**：收集交通流量大和小的圖片，包括不同類型的車輛、行人、自行車等。

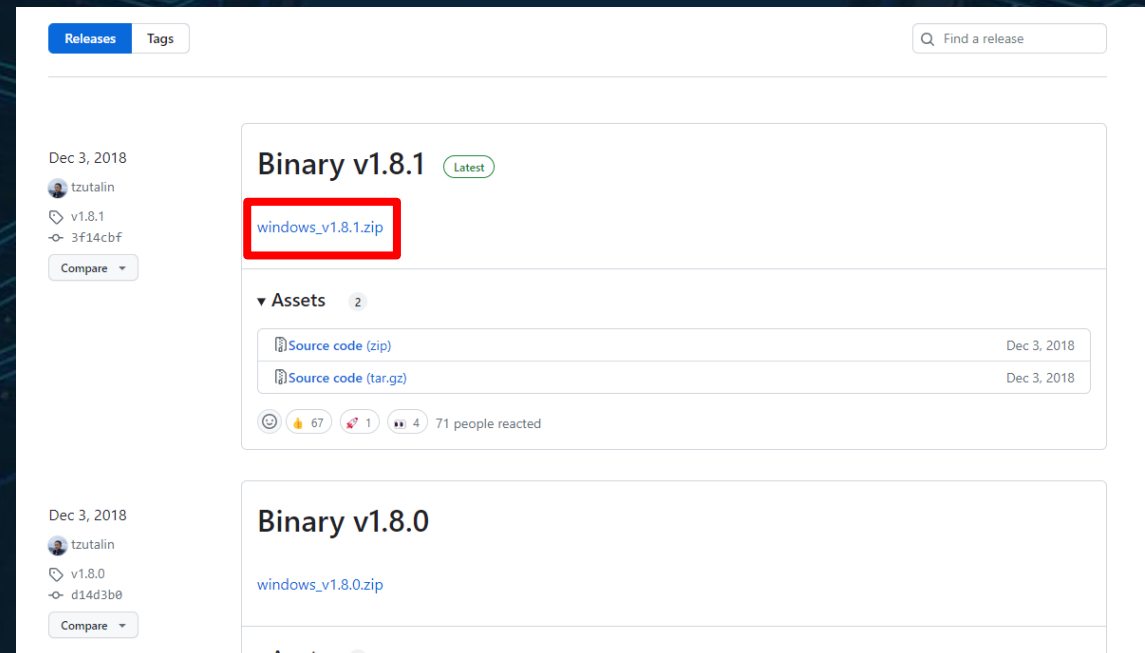


圖像標記工具

- Labellmg
- Roboflow

使用 LabelImg 圖像標記工具

- LabelImg 是一款 Open Source 軟體，專用於影像標記：<https://github.com/tzutalin/labelImg>
- 到 Release 下載 Windows 版，若要編譯需下載 Source Code。
- **注意!!! labelImg.exe 如果閃退，請確認路徑不能有中文**



使用 LabelImg 圖像標記工具

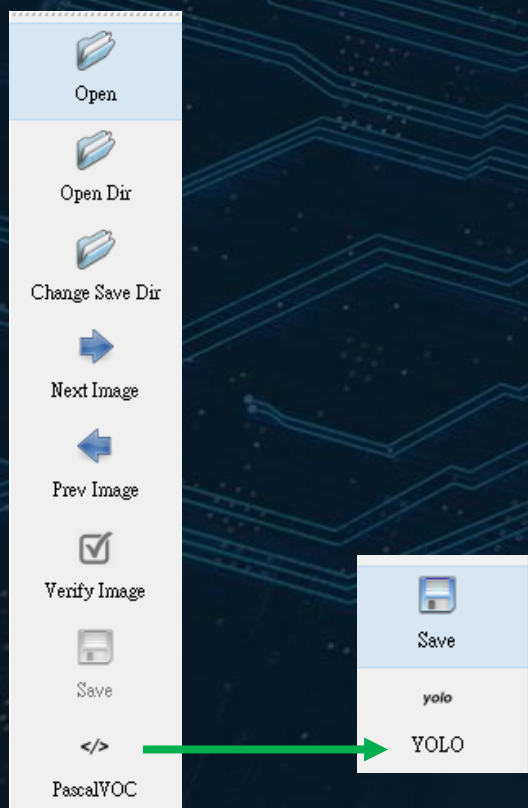
- 產生給機器學習和深度學習使用的樣本。
- 支援普遍使用的PASCAL VOC或是YOLO的資料型態。



• LabelImg程式示意圖

LabelImg 資料格式

- 資料格式可以換成 YOLO，pascal_voc 儲存的副檔名為 xml，YOLO 則為 txt。



LabelImg

- 若存成 YOLO 用的.txt 檔，格式為：
- 存成PascalVOC的.xml檔，格式為：

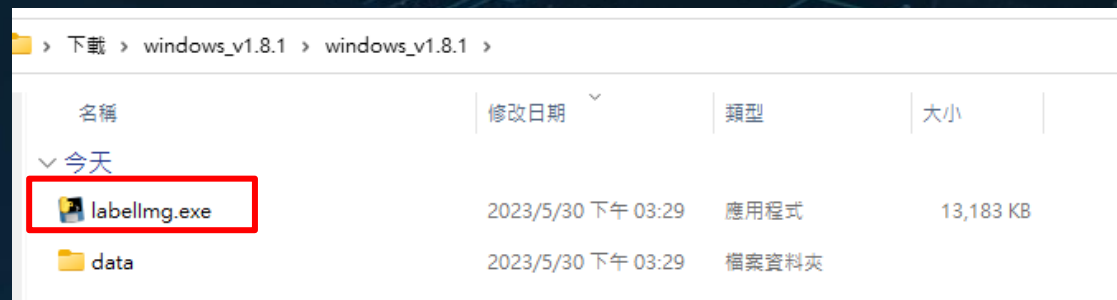
1. 物件的類別編號
2. 左上點的x座標(除以圖片寬度)
3. 左上點的y座標(除以圖片長度)
4. 物件寬度(除以圖片寬度)
5. 物件長度(除以圖片長度)

```
4 0.671089 0.541463 0.160110 0.269919
4 0.525618 0.413008 0.059469 0.100813
15 0.711345 0.791057 0.156450 0.193496
16 0.827081 0.364228 0.045746 0.084553
```

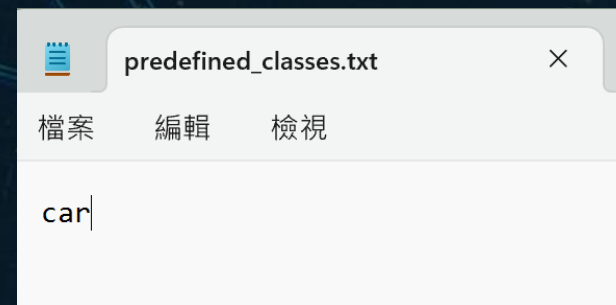
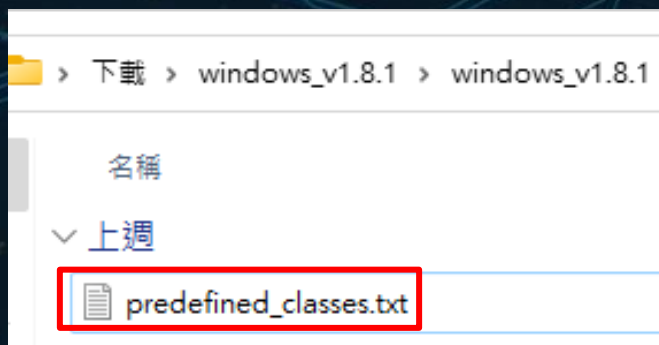
```
<?xml version="1.0"?>
- <annotation>
  <folder>Desktop</folder>
  <filename>未命名.png</filename>
  <path>C:/Users/CPS_WSN/Desktop/未命名.png</path>
+ <source>
- <size>
  <width>1093</width>
  <height>615</height>
  <depth>3</depth>
</size>
<segmented>0</segmented>
+ <object>
+ <object>
+ <object>
- <object>
  <name>speed_sign</name>
  <pose>Unspecified</pose>
  <truncated>0</truncated>
  <difficult>0</difficult>
  - <bndbox>
    <xmin>879</xmin>
    <ymin>198</ymin>
    <xmax>929</xmax>
    <ymin>250</ymin>
  </bndbox>
</object>
</annotation>
```

Labellmg 操作

- 解壓縮後先進到 data 資料夾內

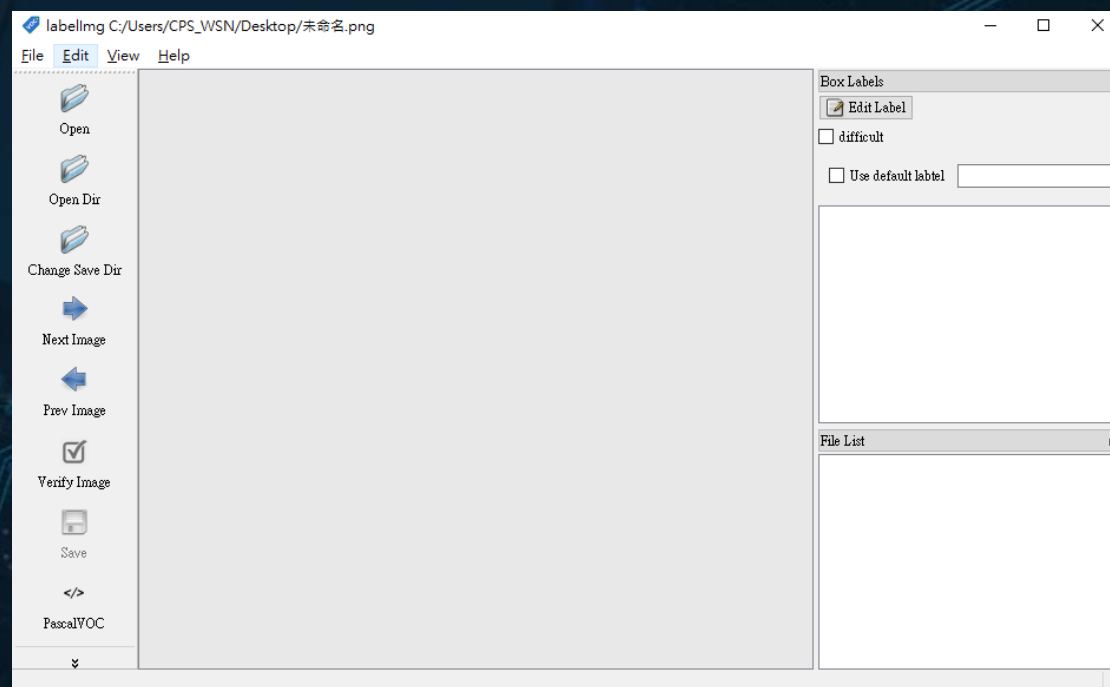
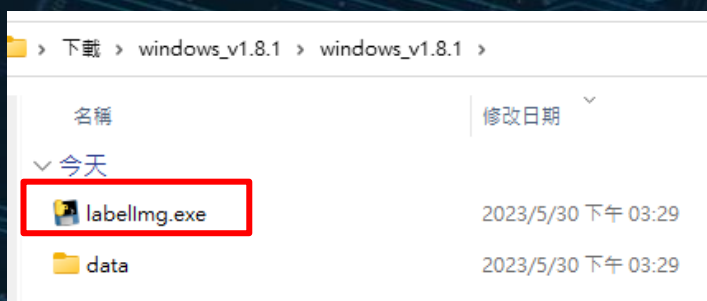


- predefined_classes.txt 為預先定義好的標籤類別 class，假設現在要標記車子的資料，則將裡面的 class 改成只有一個 car



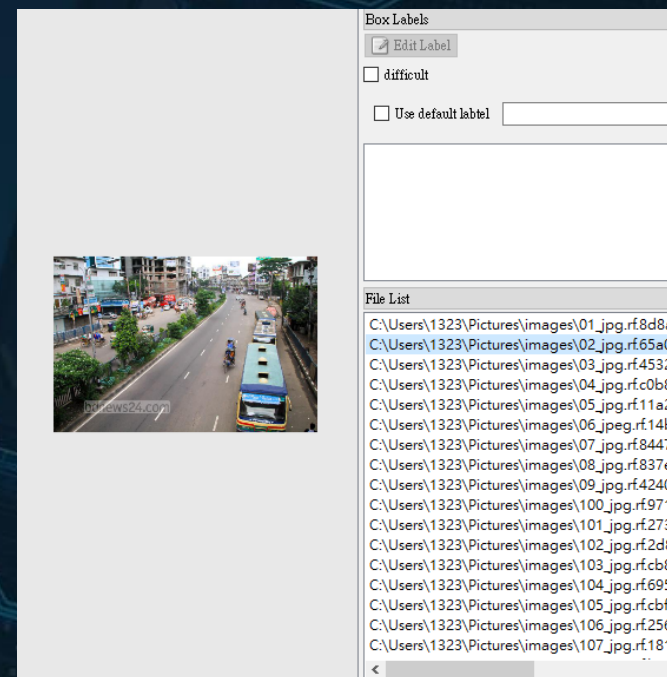
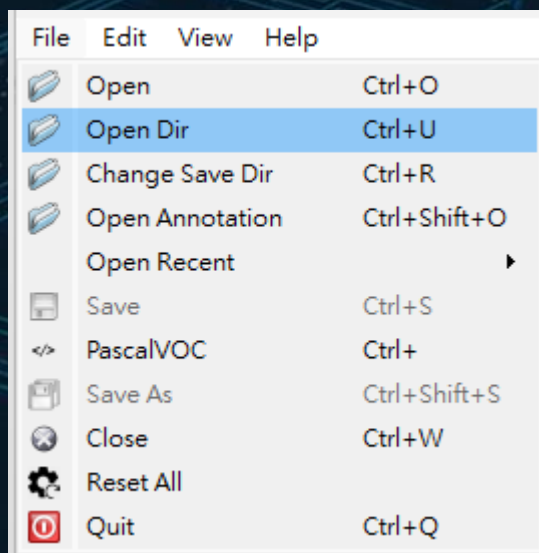
Labellmg 操作

- 改好 predefined_classes.txt 後，點選 labellmg.exe，會進入下圖畫面。



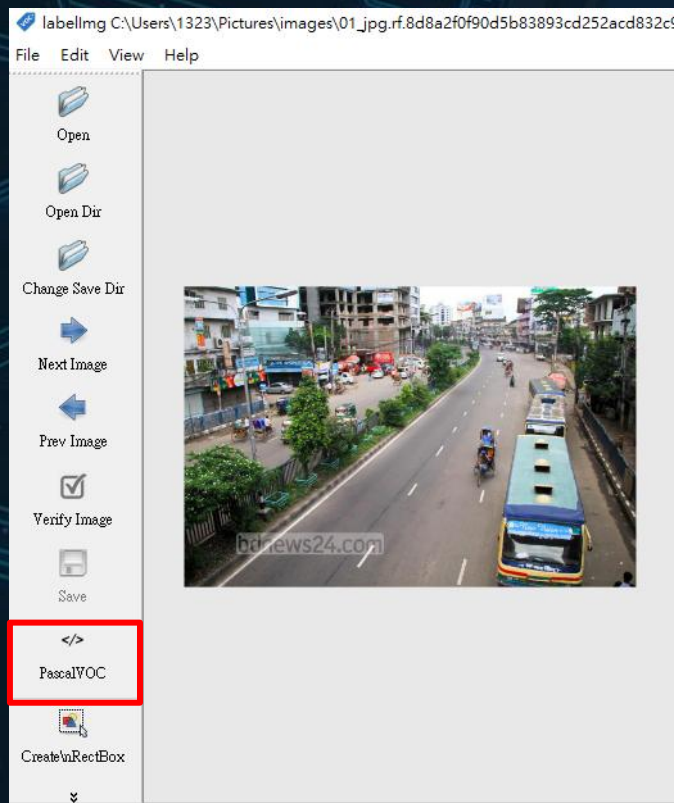
Labellmg 操作

- File > Open Dir 打開存放你要標記圖片的資料夾
- 打開後右下角的 file list 會出現該資料夾內所有的圖片名稱



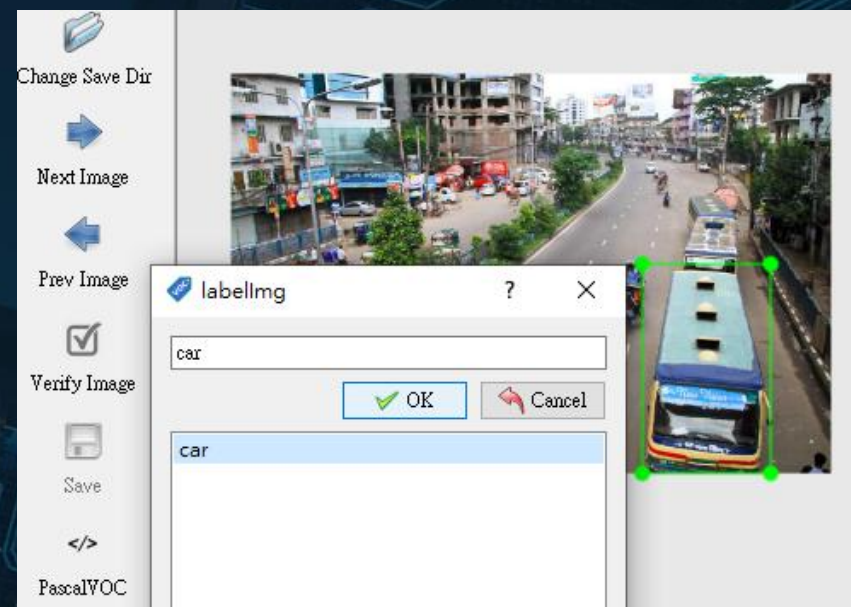
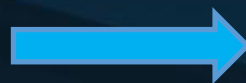
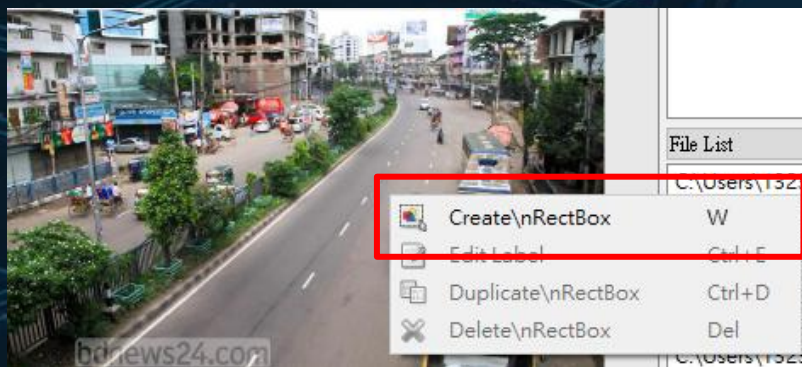
Labellmg 操作

- 右邊預設的資料格式是 pascal_voc，點擊一下切換成 YOLO



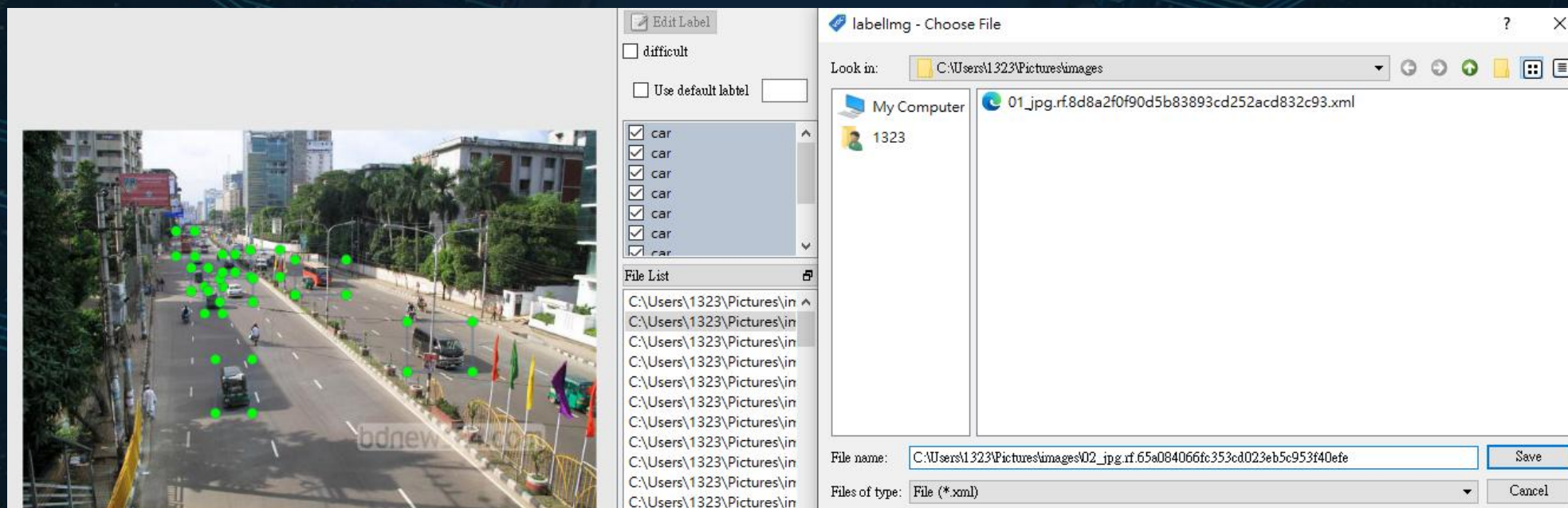
Labellmg 操作

- 點開圖右鍵，選擇 Create \nRectBox
- 框選車車，選擇標籤 car 後點 OK



Labellmg 操作

- 標記完一張圖片的所有車車之後，記得按 Ctrl S 存檔(會是一個 txt 檔)
- 所以你如果有 20 張圖片要標記，就會有 20 個 txt 檔

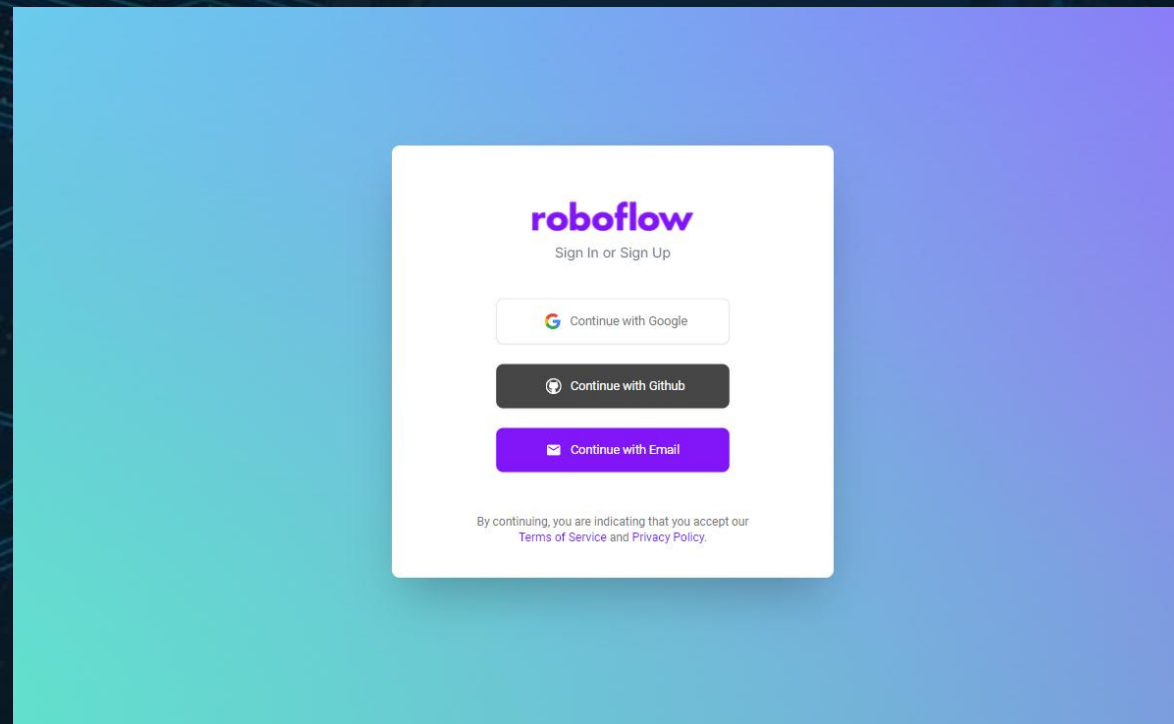


使用 Roboflow 創建和標記圖像數據集

- 本階段會介紹使用 Roboflow 平台學習創建和標記圖片資料集。
- Roboflow 是一個用於標記和預處理圖像數據的平台，它能夠幫助開發者更輕鬆地準備和管理圖像數據，用於機器學習和計算機視覺項目。
- Roboflow 提供了許多工具和功能，包括圖像標記工具、圖像預處理、數據集版本控制、數據增強和模型部署。通過 Roboflow，開發者可以更快地建立和訓練機器學習模型，從而加速其應用的開發進程。

登入 Roboflow

- 註冊帳號並登入: <https://app.roboflow.com/login>



設定工作空間 WorkSpace

- 登入後，網站會引導大家建置 workspace，名稱與選項依自己的喜好填寫即可。
- 下個頁面會詢問，是否新增協作者的 email，沒有的話點選右下方的「Skip this step」。
- 接著可以依照自己的需求選擇公開或隱藏(需要付費)資料集，並按下創建按鈕。
- 設定完工作空間之後，我們就能新增專案，請點選「Create New Project」。

WorkSpace 創建 New Project

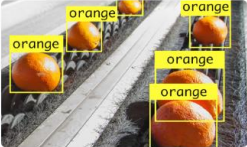
- 點選 create 創建屬於自己的圖片資料集。
- 專案類型請選擇「Object Detection(Bounding Box)」。

 **Let's create your project.**
chicken test > [New Public Project](#)

Project Name License

Annotation Group

Project Type



Object Detection
Identify objects and their positions with bounding boxes.

Best For
[Counting](#) [Tracking](#)



Classification
Assign labels to the entire image.

Classification Type
☒ Multi-Label ☐ Single-Label

Best For
[Filtering](#) [Content Moderation](#)



Instance Segmentation
Detect multiple objects and their actual shape.

Best For
[Measurements](#) [Odd Shapes](#)



Keypoint Detection
Identify keypoints ("skeletons") to subjects.

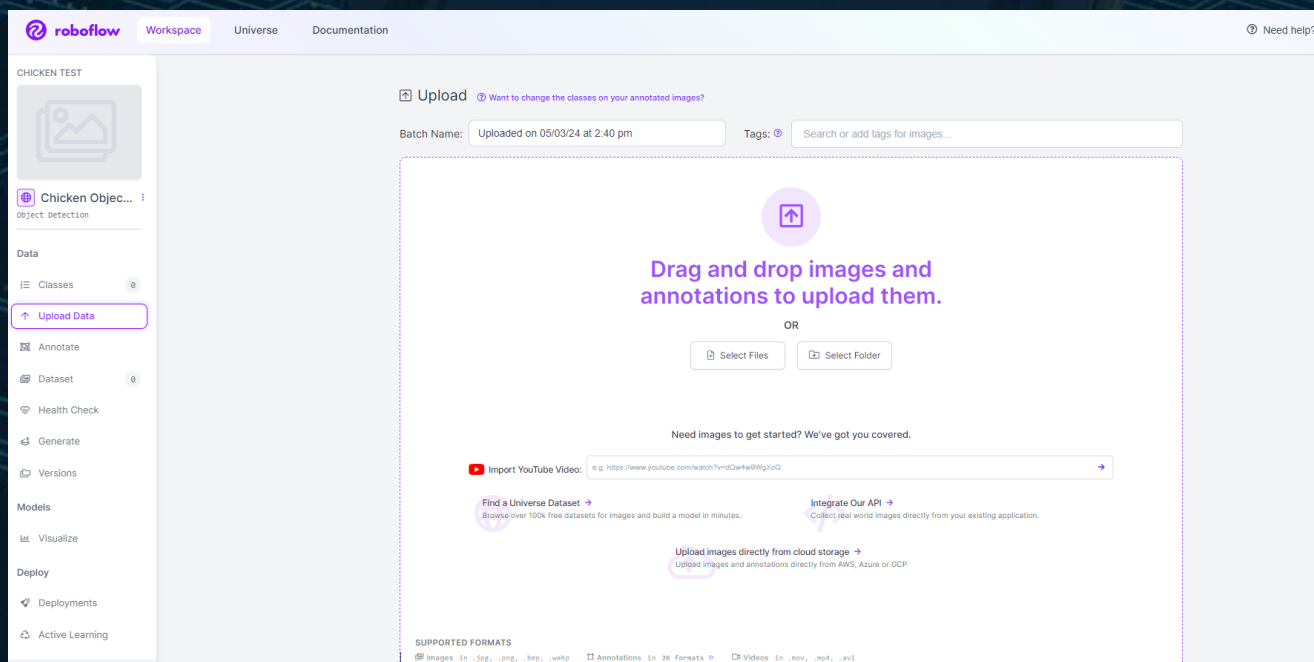
Best For
[Pose Estimation](#)

[Show More](#)

[Cancel](#) [Create Public Project](#)

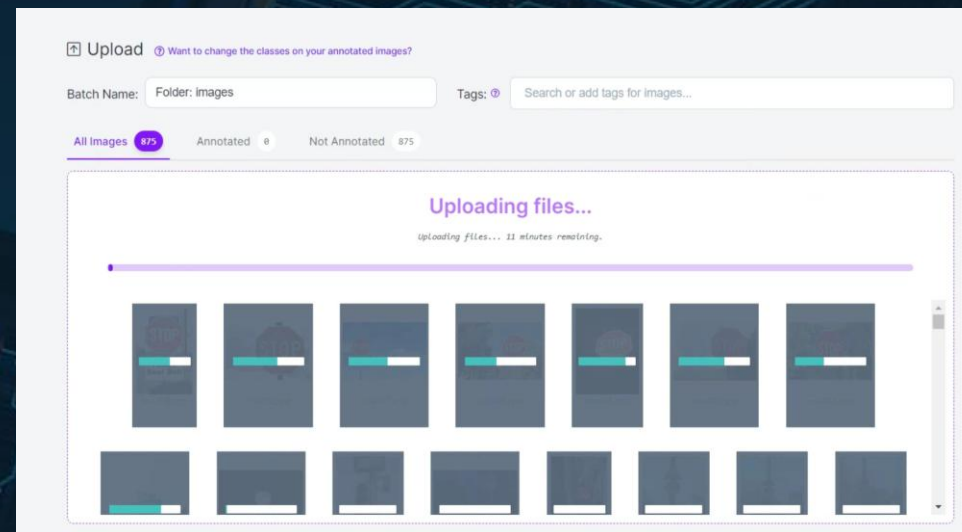
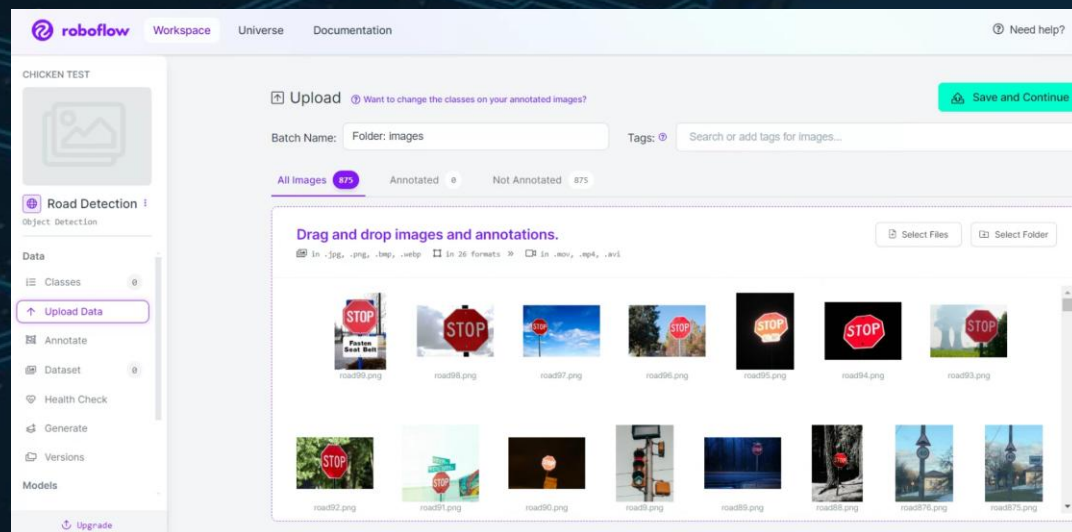
上傳欲標記的圖片檔

- 點選 Select Folder 上傳準備好的圖片資料夾(以路標資料為例)。
- 範例資料集來源: <https://www.kaggle.com/datasets/andrewmvd/road-sign-detection?resource=download>



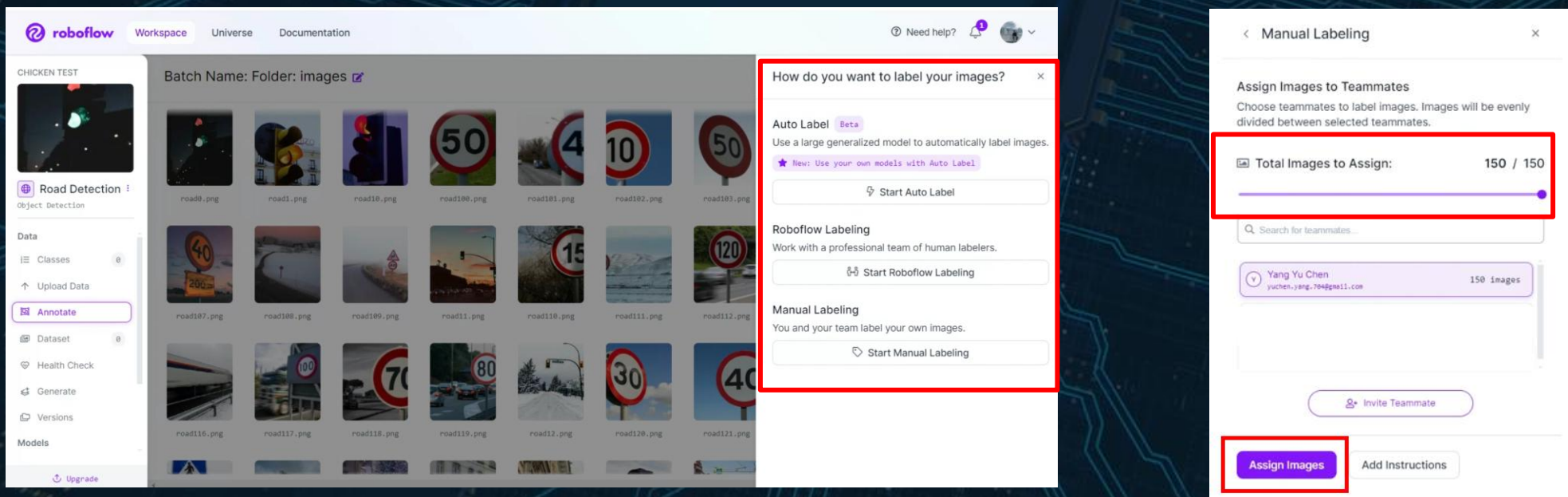
上傳欲標記的圖片檔

- 上傳完畢後，照片會顯示在視窗內，確認無誤即可點選右上角的Save and Continue 將所有照片匯入專案。



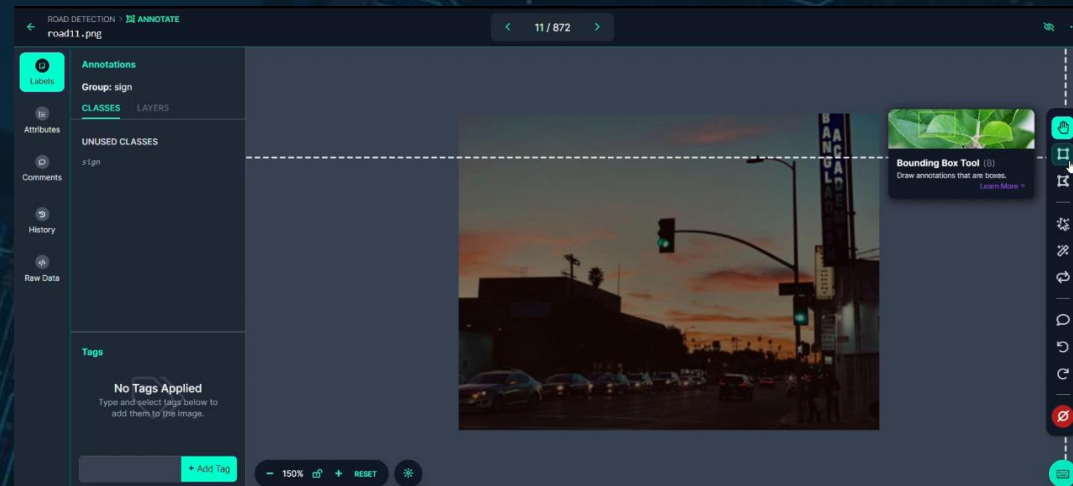
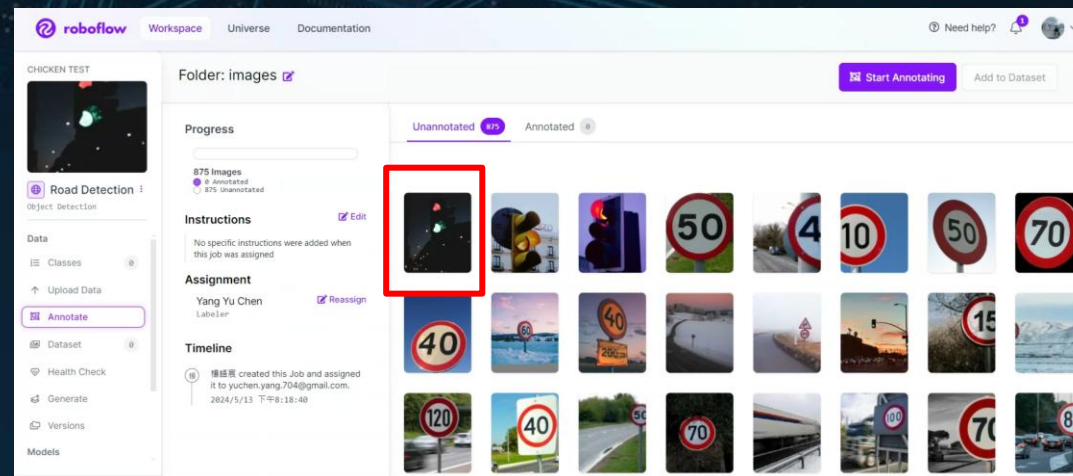
上傳欲標記的圖片檔

- 匯入完成，系統會提示是否開始標記圖片，點選 Start Manual Labeling / Assign Images 開始進行手動標記。
- 可點選 Invite Teammate 分配圖片與組員共同進行標記，也可調整一次需要標記的圖片數量。



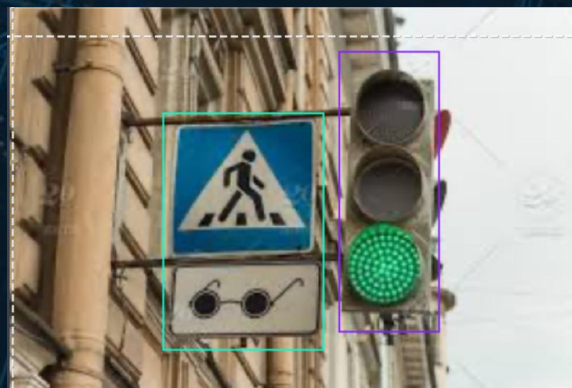
開始進行標記

- 點選一張照片，開始標記工作。
- 滑鼠點選右側工具中的「方框」，並點選照片中要框選的目標附近，拖曳滑鼠繪製方框。
- 繪製完成會需要設定此方框所代表的標籤 (class)，如影片中的「speed limit」、「traffic light」。
- 確認標記完成後繼續標記其他目標。



開始進行標記

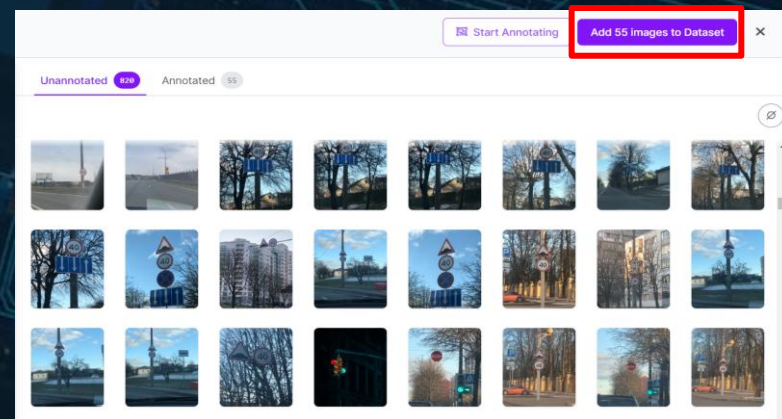
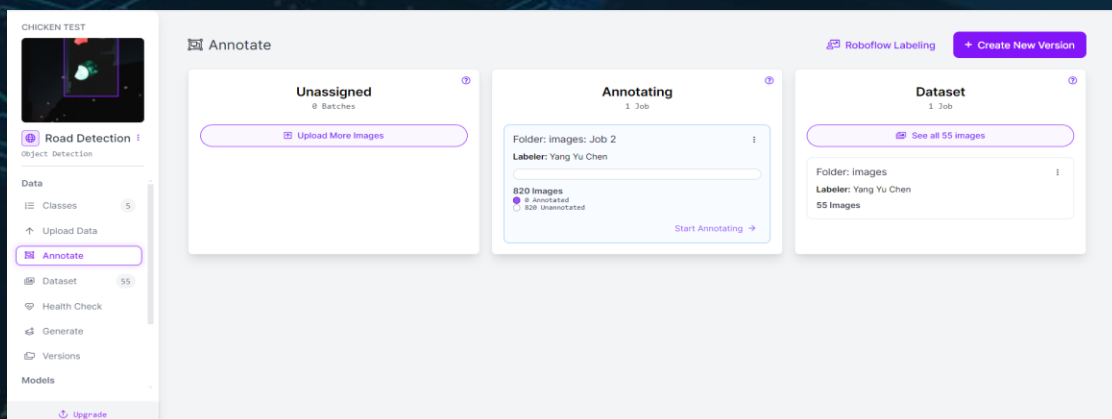
- 一張圖片若有不同物件需要標記，會需要設定不同標籤名稱。
- 不同的標籤會有不同的顏色區分，請依序標記所有的照片，雖然過程繁瑣且耗時間，但資料標記的正確性會大大影響模型的訓練，務必標記正確，且不要有過多的背景被框框包進去。
- 標記完成後，點左上角箭頭可回前一頁面。



多物件標記範例

創建資料集

- 點選 Annotating 可查看正在進行中的標記任務，Unannotated 為尚未進行標記的圖片，Annotated 為已標記過的圖片，可點選確認圖片標記的標籤是否正確。
- 標記一段落後可點選 Add Image to Dataset，將標記過的圖片加入資料集並進行分割。



創建資料集

- 資料集的部份通常會分為訓練集 Train、驗證集 Valid 與測試集 Test 三種。
 - 訓練集 (Train) - 這部分的數據用來訓練機器學習模型。訓練集佔了數據的大部分。這是因為模型需要大量的數據來學習如何正確識別或預測。
 - 驗證集 (Valid) - 這部分的數據用來在訓練過程中調整模型參數，例如決定何時停止訓練以避免過度擬合 (overfitting)。通常，驗證集會用來確保模型在新數據上也能表現良好，不僅僅是訓練集上。
 - 測試集 (Test) - 這部分的數據用於在模型訓練完全結束後進行最終的性能評估。測試集應該是模型在整個訓練過程中從未見過的數據，以此來真實地反映模型在實際應用中的表現。
- 資料比例可以藉由拖拉兩個「圓圈」做調整，依據總體資料量多寡可以自行調整，調整完畢請點選「Add Images」。

Add Images To Dataset

×

Add 55 images to dataset

[What's Train, Valid, Test?](#)

Method

Split Images Between Train/Valid/Test

Train
70%

Valid
20%

Test
10%

Image Distribution

Train: 39 images
Valid: 11 images
Test: 5 images

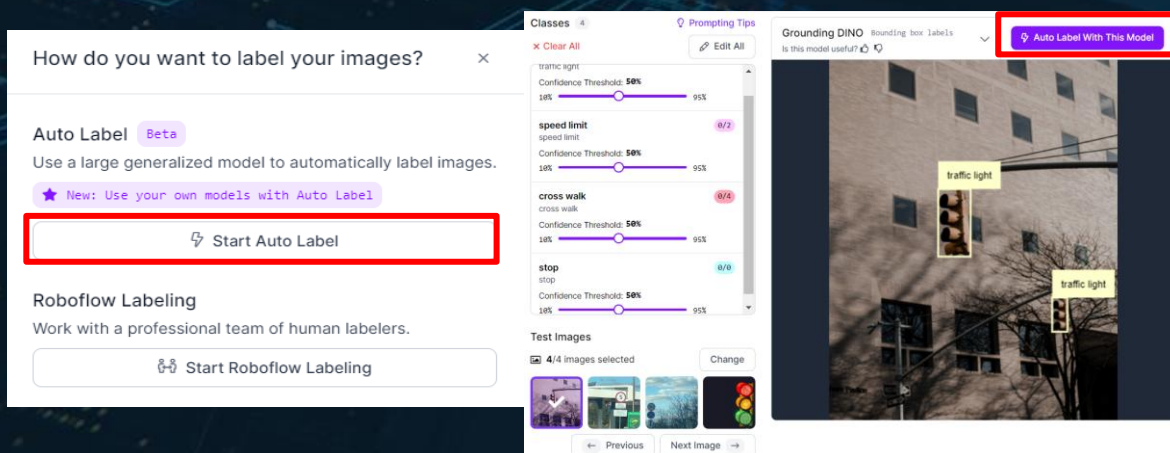
You are about to add 55 images to the dataset

820 images will be sent back as part of a new job

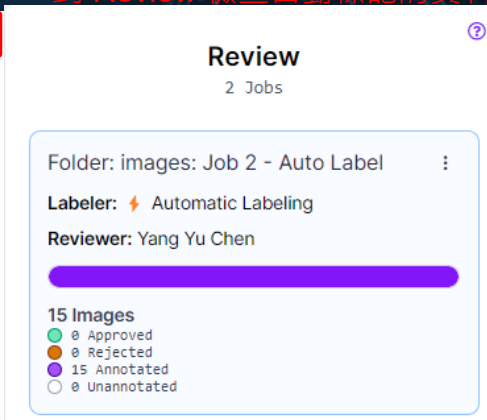
Add Images

創建資料集

- 當累積一定數量的圖片資料集(總數的10% - 20%)，並設定不同標籤(類別)後，可嘗試使用 Auto Label 功能，RoboFlow 會利用這些數據訓練其機器學習模型，自動識別並標記剩餘的未標記圖片，從而加速整個標記過程。
- Auto Label 不是完全正確的，當自動標記完成後，需要檢查並調整錯誤的標記，以確保數據的質量，也有助於模型不斷學習並改進其識別能力，保持標記的高準確性和一致性。
- 免費版的 Auto Label 有一定額度，當圖片數量過多時，可能無法自動標記完所有圖片。

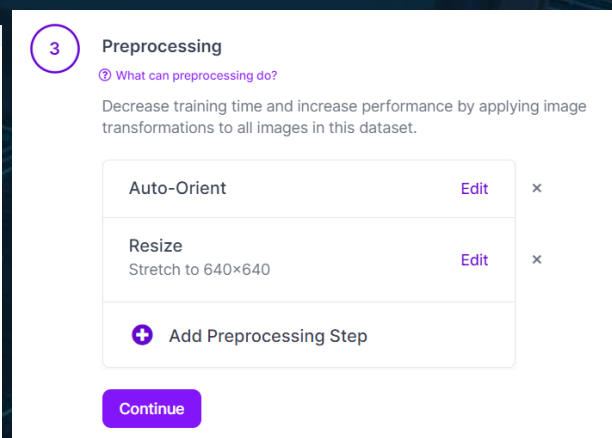
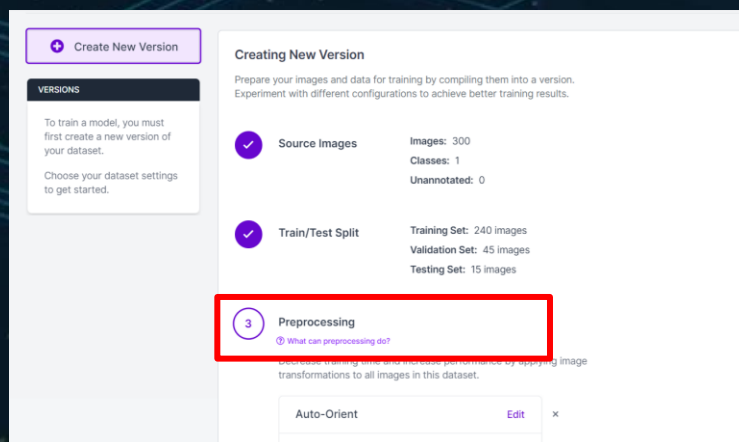


到 **Review** 檢查自動標記的資料



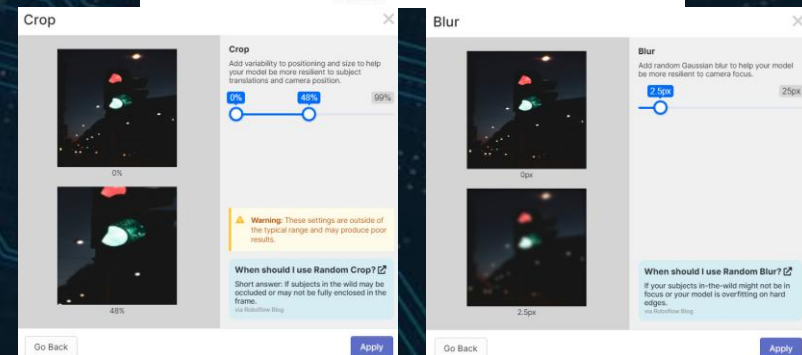
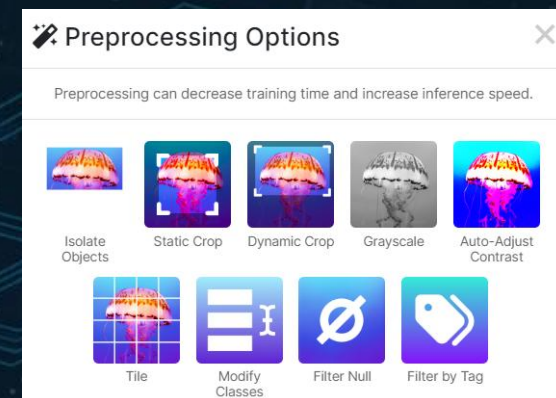
資料預處理

- 在進行資料增強前，通常需要對圖片進行基本的預處理操作，可以幫助減少訓練時間並提高模型性能。包含自動校正方向、調整大小等操作，也可依需求新增剪裁、灰度、對比度調整等步驟。
- 點選左側選單的 Generate、Preprocessing，可以快速地幫照片做一些預處理。



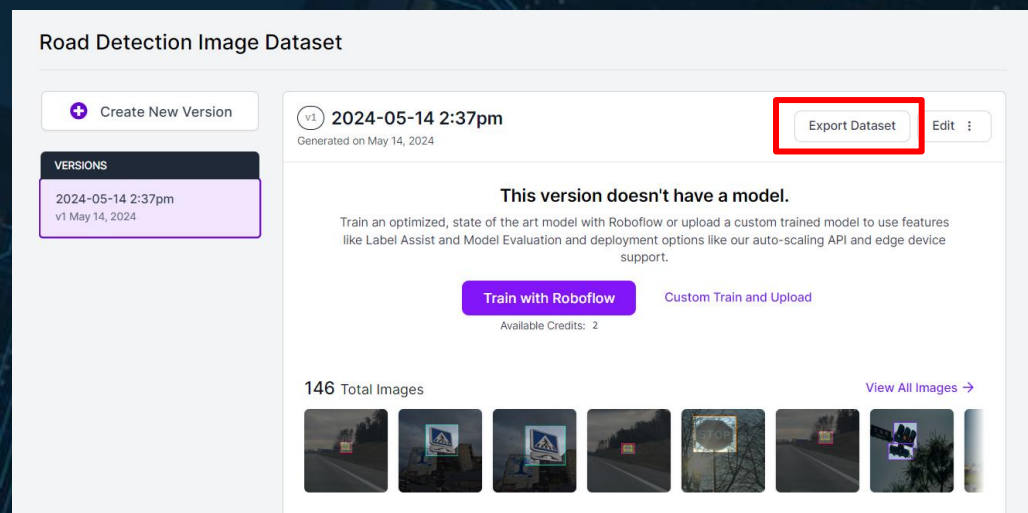
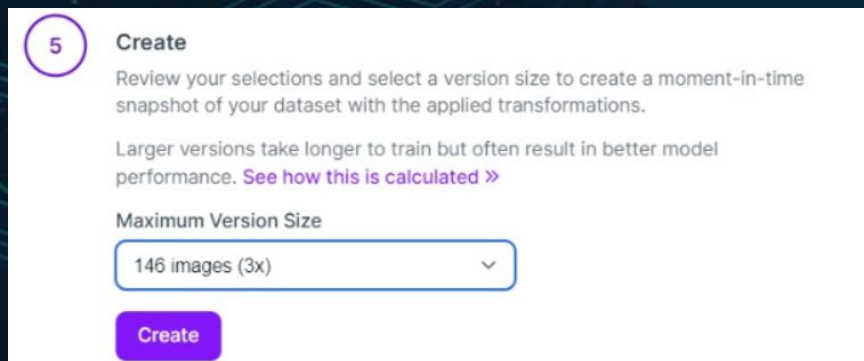
資料增強

- 點選 Continue 後，進入下一步 Augmentation，進行資料增強。
- 資料增強為在現有的訓練集上生成更多的變化，透過對現有圖片進行隨機變換(如旋轉、平移、縮放、模糊等)，以創造出新的圖片。
- 資料增強可提高模型的泛化能力、防止過擬合(overfitting)，也可應對數據不足的情況，有效擴充資料集。
- 點選 Add Augmentation Step 選取增強效果，可自行調整變化幅度。
- 完成後點選 Apply / Continue / Generate，產生增強後的資料集。



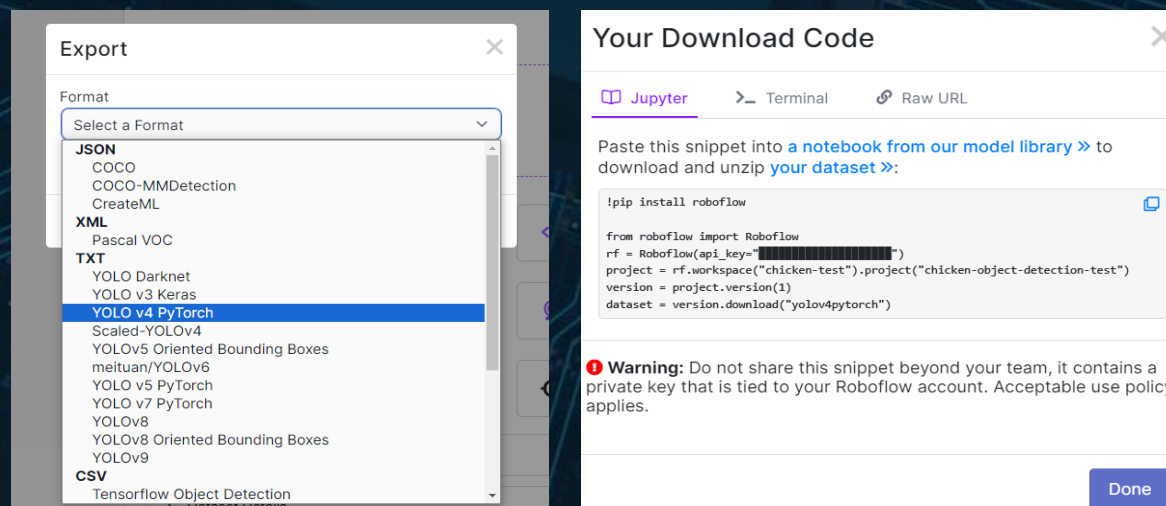
創建與導出資料集

- 最後點選 Create 創建資料集，可以透過資料增強的組合，選擇要擴充到多少張照片，免費板可以擴充到三倍。
- 完成資料集的創建後，我們就能匯出資料集，以便用於後續的訓練。點選 Export Dataset。



導出資料集

- Export 可選擇資料導出的 Format，使用不同的模型，輸入的資料格式也會不同，請對應要訓練的模型，選擇正確的模型格式，TXT 格式主要用於 YOLO 系列模型。
- 點選 Show Download Code 產生出來的 code 如下圖所示，圖中「api_key」的部份雖然是反黑的，但實際框選並複製貼上後就會顯示。



導出資料集

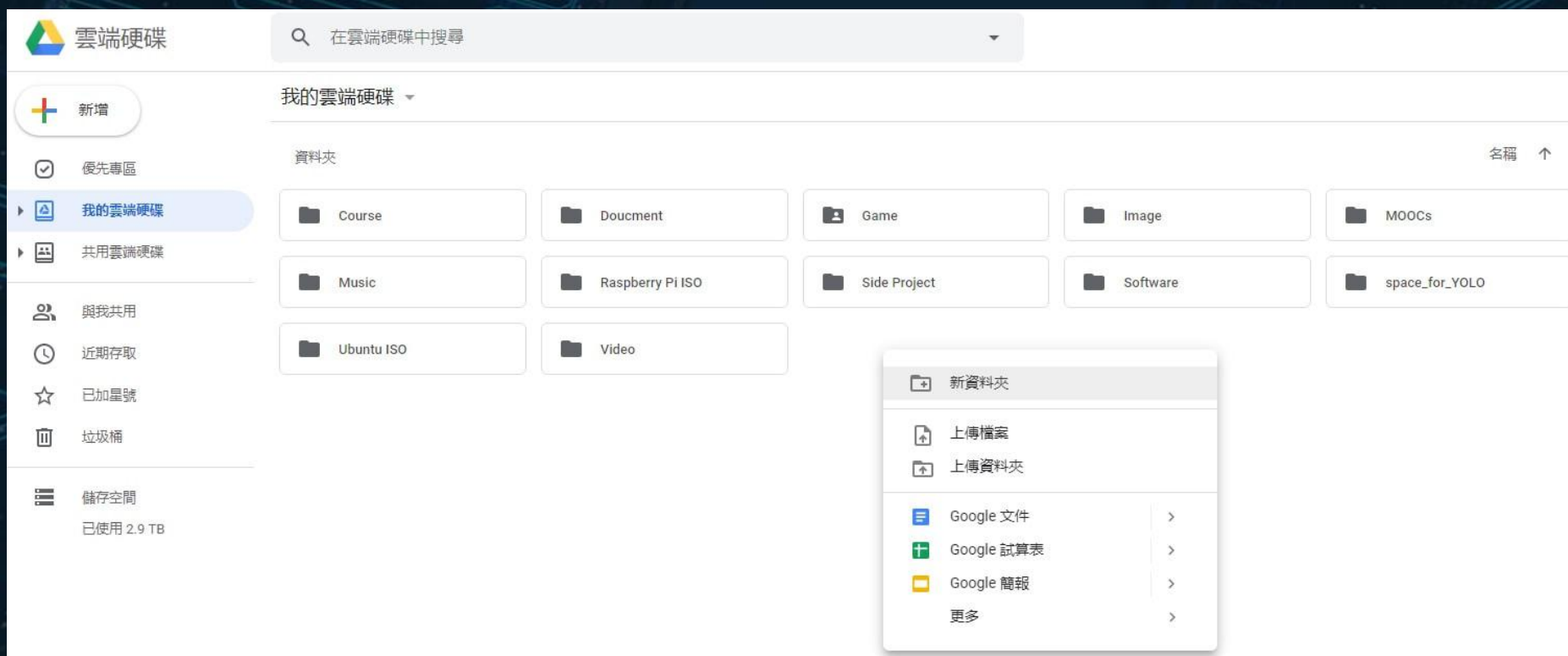
- Download Zip to computer 為將標記完成的資料集打包成 zip 檔，下載到本地端電腦。
- 適合立即需要在本地端進行資料處理或模型訓練的情況。
- 使用 Roboflow 完成了資料標註，就可以使用標記好的資料集來訓練YOLO模型。



使用 Google Colab 雲端運算平台

使用 Google Colab 雲端運算平台

- 先在個人的 Google Drive 上新增一個資料夾存放此次 project 的東西。



使用 Google Colab 雲端運算平台

- 開啟 Google Colab 然後啟用免費的GPU
- 網址：<https://colab.research.google.com/>



使用 Google Colab 雲端運算平台

- Google Colab 連結自己的 Google 雲端硬碟空間
 - 由於 Colab 的空間是一個臨時的工作空間，在建立當初是沒有任何檔案的，此時可將 Colab 連結自己的 Google 雲端硬碟空間，以利將事先準備好的 obj.names、obj.data 等檔案複製到 Colab 的工作空間，或是將訓練好的.weight 權重檔案複製回雲端。
 - 我們在自己的運算硬碟上執行，文件就不會消失。

```
#掛載自己的雲端硬碟
from google.colab import drive
drive.mount('/content/drive')

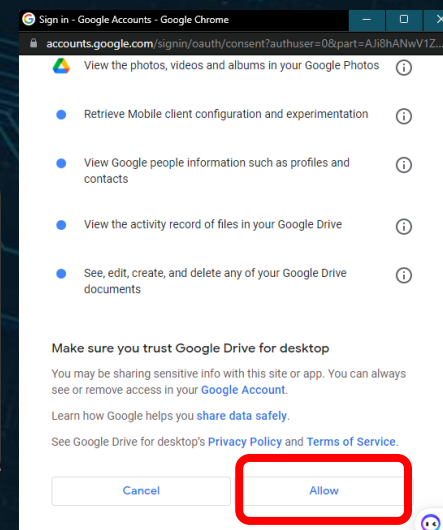
!ln -s /content/drive/My\ Drive/ /mydrive
!ls /mydrive
```

Permit this notebook to access your Google Drive files?

This notebook is requesting access to your Google Drive files. Granting access to Google Drive will permit code executed in the notebook to modify files in your Google Drive. Make sure to review notebook code prior to allowing this access.

No thanks

Connect to Google Drive



使用 Google Colab 雲端運算平台

- 掛載完自己的雲端硬碟之後，應該可以在右側的檔案找到自己雲端硬碟的位置。
- %cd 到剛剛所創建的資料夾，點選資料夾可複製路徑。
- 在資料夾內下載 darknet 並建置環境。



```
[ ] %cd /content/drive/MyDrive/yolov4_test/

↗ /content/drive/MyDrive/yolov4_test

[ ] #取得darknet資料夾並建置環境
!git clone https://github.com/AlexeyAB/darknet.git

↗ Cloning into 'darknet'...
remote: Enumerating objects: 15851, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (14/14), done.
remote: Total 15851 (delta 5), reused 12 (delta 4), pack-reused 15833
Receiving objects: 100% (15851/15851), 14.38 MiB | 11.96 MiB/s, done.
Resolving deltas: 100% (10675/10675), done.
Updating files: 100% (2056/2056), done.
```

使用 Google Colab 雲端運算平台

- 下載完成後，我們需要修改 Makefile 裡面的四個參數，分別是：
 - GPU=0 要改成 GPU=1 (開啟GPU加速)
 - OPENCV=0 要改成 OPENCV=1 (用來讀取影像、影片、畫框等功能)
 - CUDNN=0 要改成 CUDNN=1 (用於加速 tensorflow、pytorch 等深度學習框架)
 - CUDNN_HALF=0 要改成 CUDNN_HALF=1 (建構tensor核心，加速偵測物件)

```
# 修改 Makefile 裡面的四個參數
%cd darknet
!sed -i "s/GPU=0/GPU=1/" Makefile
!sed -i 's/OPENCV=0/OPENCV=1/' Makefile
!sed -i 's/CUDNN=0/CUDNN=1/' Makefile
!sed -i 's/CUDNN_HALF=0/CUDNN_HALF=1/' Makefile

📁 /content/drive/My Drive/project2_2023/darknet
```

備註：若以上指令無法更改可自 colab 左邊目錄直接打開 /content/darknet/Makefile 檔案，進行修改。

使用 Google Colab 雲端運算平台

- 測試編譯 yolov4

```
[ ] #務必確認當前的作業資料夾是在Google雲端硬碟中 (/content/drive/MyDrive/)  
#可輸入!pwd獲得當前位置  
!pwd
```

```
📁 /content/drive/MyDrive/yolov4_test
```

```
▶ #編譯 yolov4  
! cd darknet; make  
📁 gcc -Iinclude/ -I3rdparty/stb/include -DOPENCV pkg-config --cflags opencv4 2> /dev/null || pkg-config --cflags opencv -DGPU -I/usr/local/cuda/include/  
./src/connected_layer.c: In function 'forward_connected_layer_gpu':  
./src/connected_layer.c:346:11: warning: unused variable 'one' [-Wunused-variable]  
346 |     float one = 1;    // alpha[0], beta[0]  
    |           ~~~  
./src/connected_layer.c:344:13: warning: unused variable 'c' [-Wunused-variable]  
344 |     float * c = 1.output_gpu;
```

會跑一陣子

使用 Google Colab 雲端運算平台

- 下載 yolov4 Darknet 官方事先訓練好的 weight

```
[ ] #使用yolov4
! cd darknet ; ./darknet

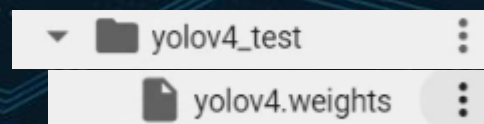
usage: ./darknet <function>

[ ] #下載 yolov4 Darknet 官方事先訓練好的 weight
!wget -N https://github.com/AlexeyAB/darknet/releases/download/darknet_v4_pre/yolov4.weights

--2024-05-29 16:50:30-- https://github.com/AlexeyAB/darknet/releases/download/darknet_v4_pre/yolov4.weights
Resolving github.com (github.com)... 20.205.243.166
Connecting to github.com (github.com)|20.205.243.166|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://objects.githubusercontent.com/github-production-release-asset-2e65be/75388965/4b8a4e00-b2d7-11eb-900f-678196af5945?X-Amz-Algorithm=AWS4-I
--2024-05-29 16:50:30-- https://objects.githubusercontent.com/github-production-release-asset-2e65be/75388965/4b8a4e00-b2d7-11eb-900f-678196af5945?X-Amz-
Resolving objects.githubusercontent.com (objects.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to objects.githubusercontent.com (objects.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 257717640 (246M) [application/octet-stream]
Saving to: 'yolov4.weights'

yolov4.weights  100%[=====>] 245.78M  119MB/s  in 2.1s

2024-05-29 16:50:33 (119 MB/s) - 'yolov4.weights' saved [257717640/257717640]
```



下載完成後資料夾會出現
yolov4.weights

使用 Google Colab 雲端運算平台

- 測試使用 YOLOv4 偵測與辨識物件
- 使用 PIL 影像處理套件開啟圖片，此為 Darknet 提供的範例圖片之一。

```
from PIL import Image  
Image.open('darknet/data/person.jpg')
```



使用 Google Colab 雲端運算平台

- 在創建的雲端資料夾下執行以下程式
- 辨識剛剛開啟的圖片，yolov4.weights 路徑會不同，記得要修改，不然會報錯。

```
! cd darknet; ./darknet detect /content/drive/MyDrive/yolov4_test/darknet/cfg/yolov4.cfg /content/drive/MyDrive/yolov4_test/yolov4.weights /content/drive/MyDrive/yolov4_test/darknet/data/person.jpg
```

改成自己的 folder/darknet/cfg/yolov4.cfg 路徑

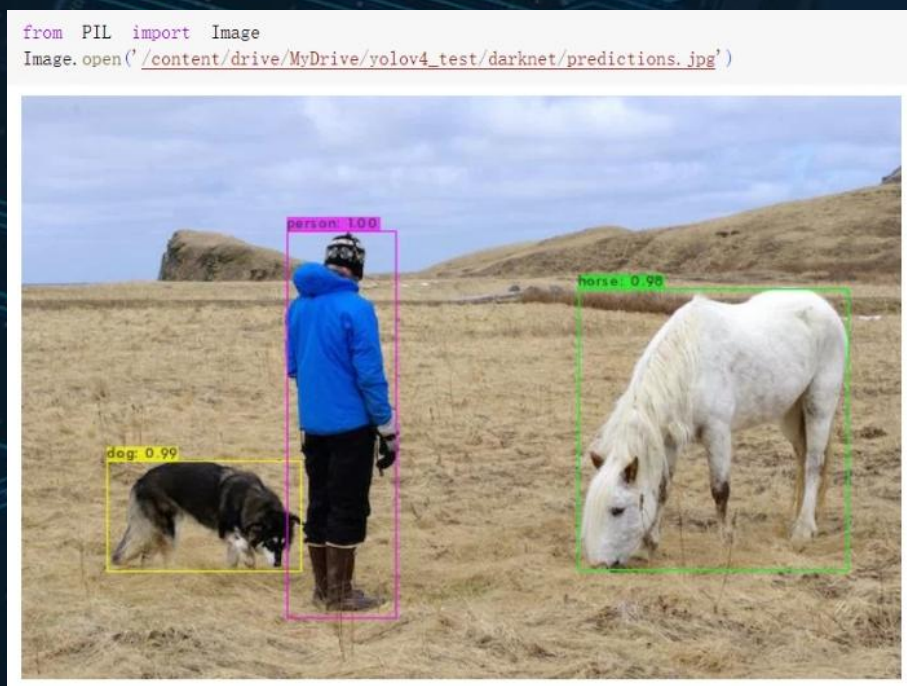
下載好的 yolov4.weights 路徑

要辨識的圖片路徑

記得修改路徑!!

使用 Google Colab 雲端運算平台

- 開啟辨識完成的圖片，有顯示對應的物件框即為測試成功。



記得修改路徑!!

使用 Google Colab 雲端運算平台

- 上述步驟測試成功後就可以訓練自己準備好的資料集，在開始之前還要準備幾個檔案
 - 在 Google Drive 創建的資料夾中上傳以下檔案：
 - 標記好的資料集 zip 檔
 - 新增兩個檔案 (可上傳後在 colab 進行編輯):
 - `obj.names` 列出標記的類別 (如:car)
 - `obj.data` 存放一些參數，物件類別數量、路徑，根據自己資料集的類別個數進行修改而得到。

```
obj.data X
1 classes=1
2 train=/content/drive/MyDrive/yolov4_test/train.txt
3 valid=/content/drive/MyDrive/yolov4_test/test.txt
4 names=/content/drive/MyDrive/yolov4_test/obj.names
5 backup=/content/drive/MyDrive/yolov4_test/backup
```

```
obj.names X
1 car
```

一開始為空，需依照後續內容修改成自己的路徑

使用 Google Colab 雲端運算平台

- 上傳後，你的雲端資料夾應該會呈現這樣：

檔案類型 ▾ 使用者 ▾ 上次修改時間 ▾		
名稱	擁有者	
 darknet	 我	
 obj.names	 我	
 obj.zip	 我	
 obj.data	 我	

使用 Google Colab 雲端運算平台

- 解壓縮 dataset，Roboflow 解壓縮完會有train/test/valid 三個資料夾

```
! unzip /path/to/car_detect.zip > data_unzip.log
```

Dataset 路徑

#查看圖片是否載入成功

```
from PIL import Image
```

```
img = Image.open('/content/drive/MyDrive/yolov4_test/test/frame_0005.jpg.rf.6b437ab7a0613048d8779ed53bdlec46.jpg')
```

```
img
```

隨機選一張資料集中的圖片查看



記得修改路徑!!

使用 Google Colab 雲端運算平台

- 產生 train.txt & test.txt (使用 RoboFlow)
- train 與 test 資料夾裡分別有對應的 jpg 圖片檔案以及 YOLO 影像標註檔案(.txt)，在訓練前我們需要把資料輸出成一個 file list (圖片的絕對路徑) 這樣跑訓練時才會讓 YOLO 知道你的檔案在哪裡，待會會用到這兩個檔案。

```
import os

def generate_file_list(image_dir, output_file):
    with open(output_file, 'w') as f:
        for root, _, files in os.walk(image_dir):
            for file in files:
                if file.endswith('.jpg'):
                    file_path = os.path.join(root, file)
                    f.write(file_path + '\n')

# 設定資料夾路徑
train_dir = '/path/to/dataset/train'
test_dir = '/path/to/dataset/test'

# 設定輸出文件名
train_output_file = '/path/to/store/train.txt'
test_output_file = '/path/to/store/test.txt'

# 生成文件列表
generate_file_list(train_dir, train_output_file)
generate_file_list(test_dir, test_output_file)
```

Dataset 解壓縮完的 train/test 資料夾路徑

設定要 output 的 train.txt / test.txt 路徑

產生 train.txt & test.txt 後

記得去更改 obj.data 的 train 與 valid 路徑

```
obj.data X
1 classes=1
2 train=/content/drive/MyDrive/yolov4_test/train.txt
3 valid=/content/drive/MyDrive/yolov4_test/test.txt
4 names=/content/drive/MyDrive/yolov4_test/obj.names
5 backup=/content/drive/MyDrive/yolov4_test/backup
```

記得修改路徑!!

使用 Google Colab 雲端運算平台

- 產生 train.txt (使用 **Labellmg**)，可在本地端電腦先執行此步驟。
- 該程式會根據資料集內的檔案內容自動產生所有路徑的 train.txt 檔，並存在當前的路徑中。
- 將生成的 train.txt 上傳到雲端資料夾中，並去更改 obj.data 的 train 與 valid 路徑。

```
import os
image_files = []
# 切換到 "data/obj" (自定義資料集的路徑)
os.chdir(os.path.join("data", "obj"))
# 獲取當前目錄中的所有文件名
for filename in os.listdir(os.getcwd()):
    # 如果文件名以 ".png" 結尾
    if filename.endswith(".png"):
        # 將文件的路徑加入到 image_files 列表中
        image_files.append("data/obj/" + filename)
# 切換回上一級目錄
os.chdir("../")
# 打開 "train.txt" 文件，準備寫入
with open("train.txt", "w") as outfile:
    # 將每個圖片文件的路徑寫入文件中
    for image in image_files:
        outfile.write(image)
        outfile.write("\n") # 每個路徑後寫入一個換行符
# 關閉文件
outfile.close()
# 切換回上一級目錄
os.chdir("../")
```

obj.data ×

```
1 classes=1
2 train = data/train.txt
3 valid = data/train.txt
4 names = data/obj.names
5 backup=backup
```

記得修改路徑!!

使用 Google Colab 雲端運算平台

- 修改 yolov4-custom.cfg
- 先到 darknet/cfg 資料夾裡面複製一份 yolov4-custom.cfg

```
[ ] # 複製 yolov4-custom.cfg  
! cp /content/drive/MyDrive/yolov4_test/darknet/cfg/yolov4-custom.cfg /content/drive/MyDrive/yolov4_test
```

要複製的 cfg 檔

目的地資料夾

依講義內容進行修改:

1. net

- 變更 batch、subdivisions, 若已經是變更後的資料則不用變更
- 變更 max_batches = class * 2000
- 變更 steps = max_batches * 0.8, 0.9

記得修改路徑!!

2. 三個 [yolo] 區塊的前一個 [convolution] 區塊的 filter 改成 (classes + 5) x 3 有三個地方須修改

- filters = (classes + 5) x 3
- classes 改成 class 數量

Note: 考慮到Colab免費算力的限制, 使用yolov4-tiny來訓練會比較快速

- yolov4.weights -> yolov4-tiny.conv.29
- cfg/yolov4-custom.cfg -> cfg/yolov4-tiny-custom.cfg



參數解釋

1. batch 和 subdivisions

- batch：每次訓練迭代時，網絡將處理的圖像數量。常設為 64，但如果顯示卡內存不足，可以根據需要減小 subdivisions。
- subdivisions：將 batch 分成小的子批次進行處理，可以有效減少顯示卡內存的使用。當內存不足時，可以增加 subdivisions 的值（例如設為 32 或 64）來解決 "out of memory" 問題。
- 調整意義：減小 subdivisions 可以提高訓練速度，但需要更高的 GPU 記憶體。增加 subdivisions 則會減慢訓練速度，但降低內存消耗。



2. max_batches 和 steps

- max_batches：訓練的最大迭代次數，通常設為 類別數量 * 2000，如果資料集只有一個類別，則設為 2000。這決定了模型訓練的總迭代次數。
- steps：學習率衰減的步驟，通常設為 max_batches 的 80% 和 90%，例如 steps = 16000, 18000。這樣設定會在訓練過程中對學習率進行調整，有助於避免過擬合，並穩定模型的收斂過程。
- 調整意義：max_batches 需要根據資料集的類別數來設置，過少可能導致模型未訓練充分，過多則可能導致過擬合。steps 控制學習率的衰減，從而幫助模型在後期進行細緻調整，減少震盪。



3. filters 和 classes

- classes：決定模型要識別的物體類別數量，這個值必須和你的資料集中的類別數相匹配。如果資料集有 3 類物體，則設為 3。
- filters：每個卷積層的過濾器數量，通常需要根據 classes 來計算，公式為 $(classes + 5) * 3$ 。這個數字決定了每層卷積層的特徵提取能力。
- 調整意義：classes 必須和資料集中的物體類別數量一致。filters 與 classes 相關，並且影響到模型能夠學習到的特徵數量和層的深度。如果設置不正確，可能會導致模型無法正確預測物體類別和位置。

使用 Google Colab 雲端運算平台

- yolov4_custom.cfg 在 [darknet/cfg/yolov4-custom.cfg](#) 的基礎上，根據自己資料集的類別個數進行修改得到。
- 修改一：
 - batch = 64, subdivisions = 16 先不變，當訓練運行顯示 “out of memory” 時，可以嘗試將subdivisions 升至 32 或 64。
 - max_batches = ? 為自己資料集種類個數 * 2000，如果有一類即為 2000
 - Steps = ?, ? 分別為max_batches 的 80% 和 90%

記得修改參數!!

(可依據自己專案情況決定增加or減少訓練迭代次數)

Note : 考慮到Colab免費算力的限制，使用yolov4-tiny來訓練會比較快速

- yolov4.weights -> yolov4-tiny.conv.29
- cfg/yolov4-custom.cfg -> cfg/yolov4-tiny-custom.cfg

```
1 [net]
2 # Testing
3 #batch=1
4 #subdivisions=1
5 # Training
6 batch=64
7 subdivisions=32
8 width=608
9 height=608
10 channels=3
11 momentum=0.949
12 decay=0.0005
13 angle=0
14 saturation = 1.5
15 exposure = 1.5
16 hue=.1
17
18 learning_rate=0.001
19 burn_in=1000
20 max_batches = 2000
21 policy=steps
22 steps=1600,1800
23 scales=.1,.1
24
```

使用 Google Colab 雲端運算平台

- 修改二：
 - $\text{Filters} = (5 + \text{classes}) \times 3$ 如資料集種類個數 classes 為 1，則 $\text{filters} = 18$ 。
 - $\text{classes} = N$ (N 為 class 個數)
 - 共要修改三個段，如下圖所示： 記得修改參數!!

```
959 [convolutional]
960 size=1
961 stride=1
962 pad=1
963 filters=18
964 activation=linear
965
966
967 [yolo]
968 mask = 0,1,2
969 anchors = 12, 16, 19, 36, 40
970 classes=1
971 num=9
972 jitter=.3
973 ignore_thresh = .7
974 truth_thresh = 1
```

```
1047 [convolutional]
1048 size=1
1049 stride=1
1050 pad=1
1051 filters=18
1052 activation=linear
1053
1054
1055 [yolo]
1056 mask = 3,4,5
1057 anchors = 12, 16, 19, 36,
1058 classes=1
1059 num=9
1060 jitter=.3
1061 ignore_thresh = .7
1062 truth_thresh = 1
```

```
1135 [convolutional]
1136 size=1
1137 stride=1
1138 pad=1
1139 filters=18
1140 activation=linear
1141
1142
1143 [yolo]
1144 mask = 6,7,8
1145 anchors = 12, 16, 19, 36, 40
1146 classes=1
1147 num=9
1148 jitter=.3
1149 ignore_thresh = .7
1150 truth_thresh = 1
```

使用 Google Colab 雲端運算平台

- 建立 backup 資料夾儲存權重檔，obj.data 的 backup 記得改成自己的路徑。

```
[ ] # 創建 backup  
!mkdir -p /content/drive/MyDrive/yolov4_test/backup
```

- 使用設定好的檔案開始進行訓練與辨識，訓練大概要 1-2 小時。

在train自己的model時，須有一個預訓練權重檔案當基底

```
! darknet/darknet detector train /content/drive/MyDrive/yolov4_test/obj.data /content/drive/MyDrive/yolov4_test/yolov4-custom.cfg /content/drive/MyDrive/yolov4_test/yolov4.weights -dont_show -map -clear
```

自己的 obj.data 路徑 修改好的 yolov4-custom.cfg 路徑 載好的 yolov4.weights 路徑

- 如果訓練時因不明原因中斷，此後想要在上次訓練權重的基礎上繼續訓練可採用以下指令：

```
! darknet/darknet detector train /path/to/obj.data /path/to/yolov4-custom.cfg /path/to/backup/yolov4_custom_last.weights -dont_show
```

記得修改路徑!!

使用 Google Colab 雲端運算平台

- 訓練完成後，backup 資料夾會產生權重檔。
- weights 下載好後，可以自行找一張圖片放入資料夾內進行辨識，使用：

```
!./darknet detect <path of .cfg file> <path of .weights file> <path of picture>
```

這個指令進行辨識，辨識的結果會以 "predictions.jpg" 存在跟 darknet 同一個目錄底下，為了可以直接 show 在 notebook 上面，我們可以多寫一個 imshow 的 function 來實現。

Note : 考慮到Colab免費算力的限制，使用yolov4-tiny來訓練會比較快速

- yolov4.weights -> yolov4-tiny.conv.29
- cfg/yolov4-custom.cfg -> cfg/yolov4-tiny-custom.cfg

使用 Google Colab 雲端運算平台

- 辨識一張新的圖片：

```
# 辨識圖片  
from PIL import Image  
Image.open('/path/to/picture')
```



記得修改路徑!!

yolov4-custom.cfg 路徑

圖片路徑

```
! cd darknet; ./darknet detector test /path/to/obj.data /path/to/yolov4-custom.cfg /path/to/yolov4-custom_last.weights /path/to/picture
```

自己的 obj.data 路徑

backup 裡面的 weights

使用 Google Colab 雲端運算平台

- 開啟辨識結果 predictions.jpg :

```
from PIL import Image  
Image.open('/content/drive/MyDrive/yolov4_test/darknet/predictions.jpg')
```



記得修改路徑!!

使用 Google Colab 雲端運算平台

- 在 Google Drive 資料夾中加入要辨識的影片後進行辨識：

```
!cd darknet; ./darknet detector demo /path/to/obj.data /path/to/yolov4-custom.cfg /path/to/backup/yolov4-custom_last.weights /path/to/carvideo.mp4 -dont_show -out_filename /path/to/result.mp4 -ext_output
```

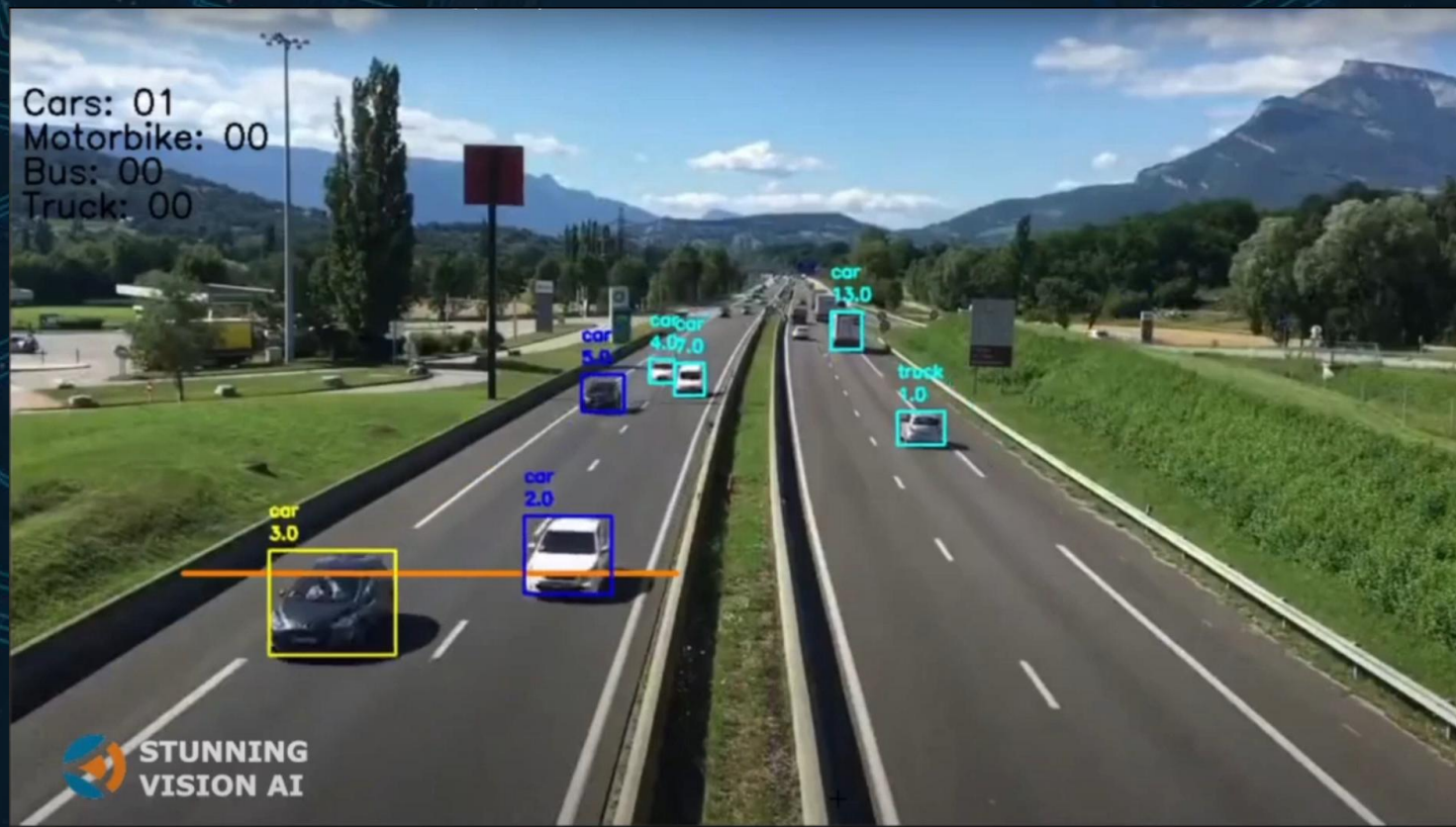
影片路徑

輸出成果影片路徑

記得修改路徑!!

使用 Google Colab 雲端運算平台

- YOLOv4 深度學習模型辨識車輛影片範例



使用 Google Colab 雲端運算平台

- 使用 Google Colab 需要注意的事項：
 - 由於 Google Colab 提供的是免費使用的運算平台，資源總是有限，因此若是訓練時間置過久會可能會主動中斷連結，這時候已執行的結果都會被清空。
 - Colab 最長的執行時間為 12 小時 (助教連T4最長只有5小時左右)，但訓練 YOLO 可能長達數天以上，因此可參考下方參考資料建立一個專用的 Colab disk 空間，讓每次重新執行 Colab 時，不會遺失訓練結果，更可以很快設定好訓練環境並從上次中斷的地方繼續訓練。
請參考：<https://makerpro.cc/2020/02/use-google-colab-to-train-yolo/>
- (推薦)可以使用自己的顯示卡進行運算

專案實作-物件辨識(影片與程式部分)

- 專案項目:YOLOv4物件辨識(可以使用其他YOLO版本!)
- 尋找兩個車輛行進中的影片，將能成功使用 yolo 辨識行進中車輛的影片上傳至 youtube，並將影片連結貼至小組報告。
- 本專案實作。(70%)
 - 資料收集。(30%)
 - 自製資料集，並提供資料集連結(雲端)。**2人組200張以上、3人組300張以上**
 - 模型訓練。(20%)
 - (推薦)可以使用自己的顯示卡進行運算，可以不用使用Colab。
 - 辨識車輛。(20%)
- 撰寫操作過程學習心得並以小組報告(PDF)、實作code、個人報告(PDF)、影片連結(30%)



小組報告格式規定

- 實驗內容
 - 此次實驗需要做什麼？
- 實驗過程及結果
 - 預期實驗的結果
 - 實際上的結果
 - 遇到的問題&問題怎麼解決？
- 本次實驗過程說明與解決方法(為上述”實驗過程及結果”的摘要、總結)
 - 實驗過程
 - 解決方法
- 成果展示
 - 影片連結展示
- 分工
 - 說明小組成員分工內容與比例。



個人報告內容

- 實驗心得：（150字）
 - 此次實驗的目的？
 - 在實驗上遇到的問題？怎麼解決？
 - 實驗結果跟你預期的內容是否一致？
- 組員貢獻度及工作內容：
 - 陳大明：33 %，資料查詢、文書處理、實驗實作。
 - 陳忠義：33 %，實驗設計、程式規劃、測試與除錯。
 - 陳小美：33 %，小組報告、買飲料、買午餐。



專案繳交規則

- 小組報告繳交期限:115/05/08 23:59
- 個人報告繳交期限:115/05/08 23:59(以I學園上傳時間為基準)
- 補交規則
 - 超過正常繳交期限一周成績**打9折**，每再遲交一週成績**打8折**
(最多遲交兩週)